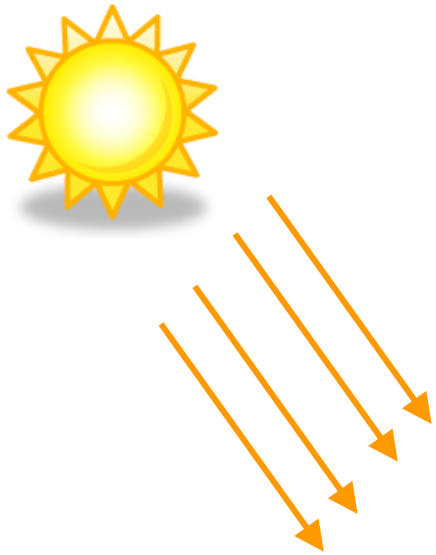
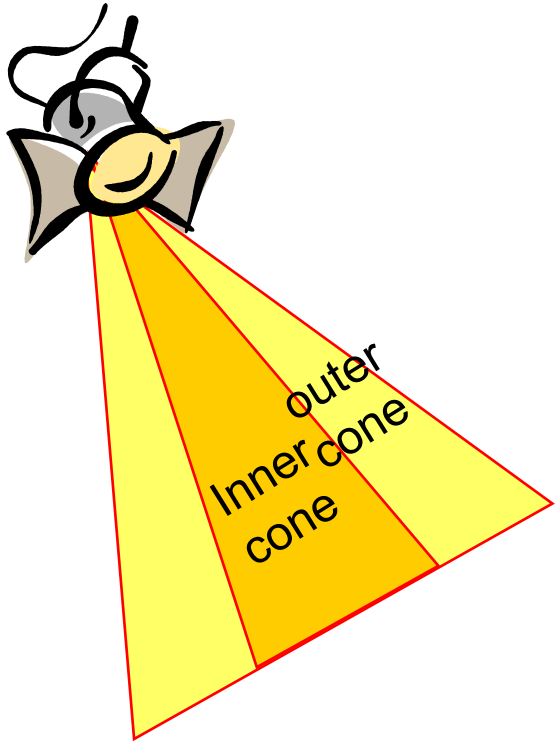
Illumination

Light Sources



Directional Light
(Infinitely far away)

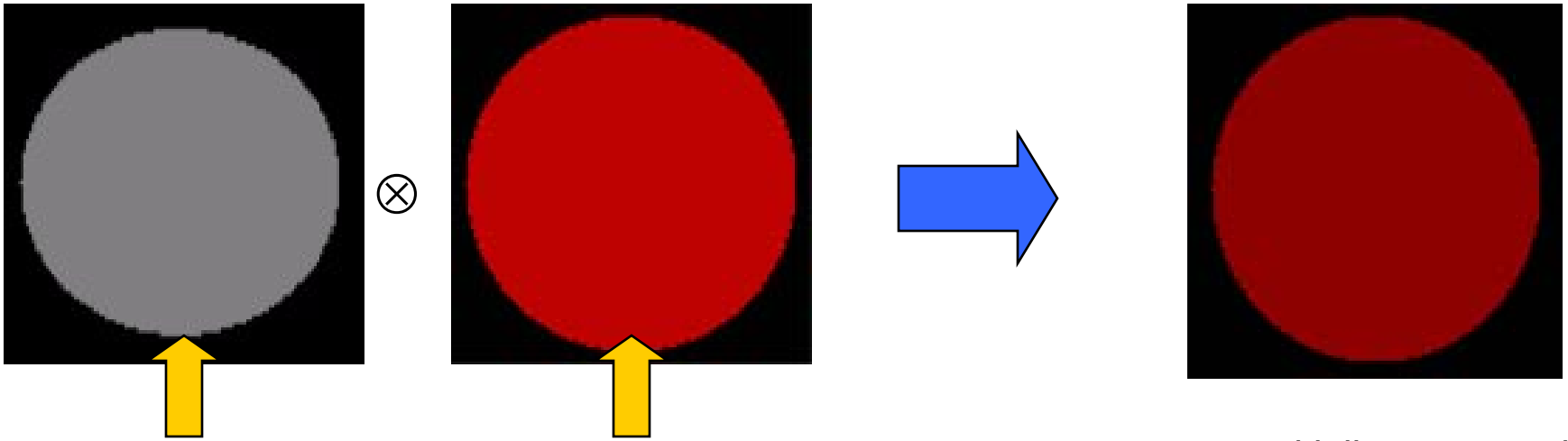
Point Light
(Emit in all directions)



Spot Light
(Emit within a cone)

Ambient Lighting

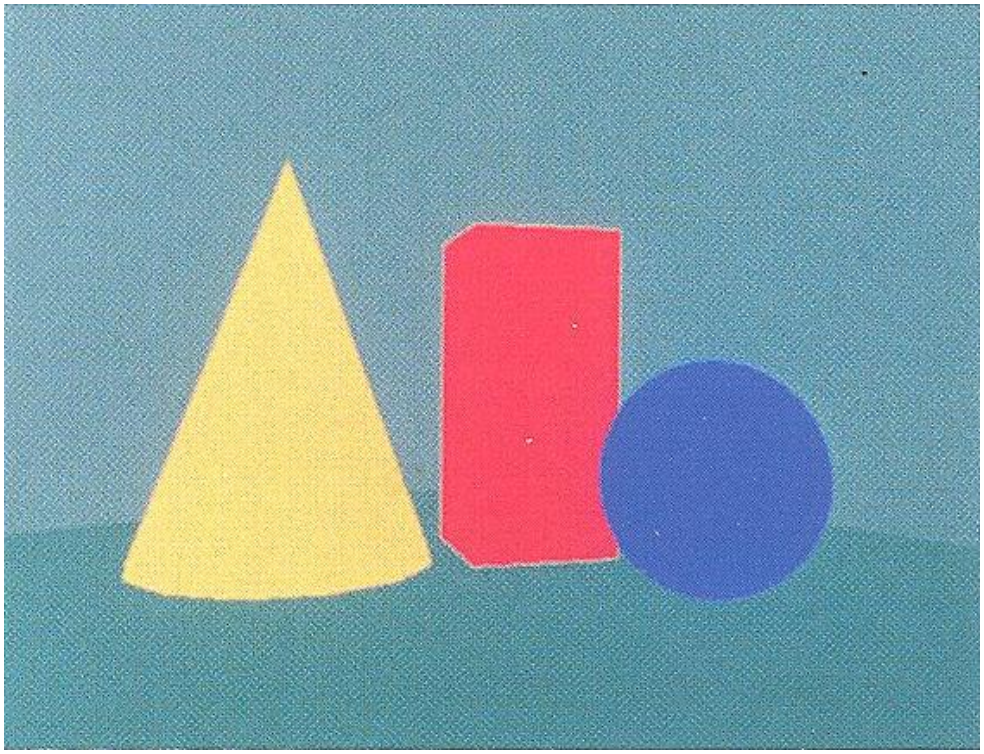
- Not created by any light source
- A constant lighting from all directions
- Contributed by scattered light in a surrounding



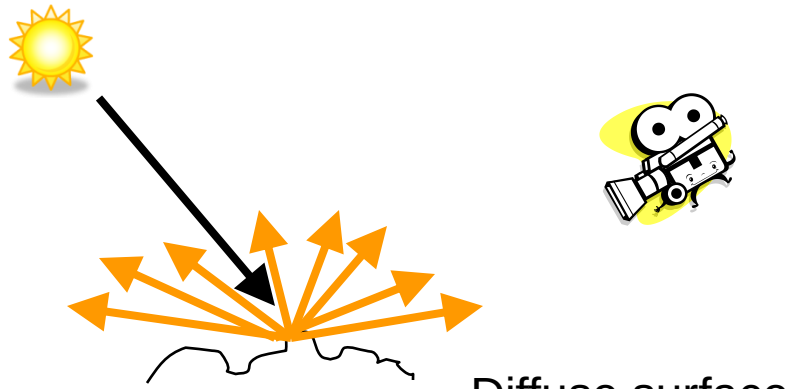
RGB multiplies separately

$$C_{ambient} = M_{ambient} \otimes L_{ambient}$$

Ambient Lighting

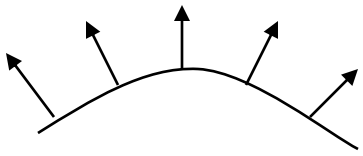


Diffuse Lighting



- Assume light bounces in all directions

Diffuse Lighting

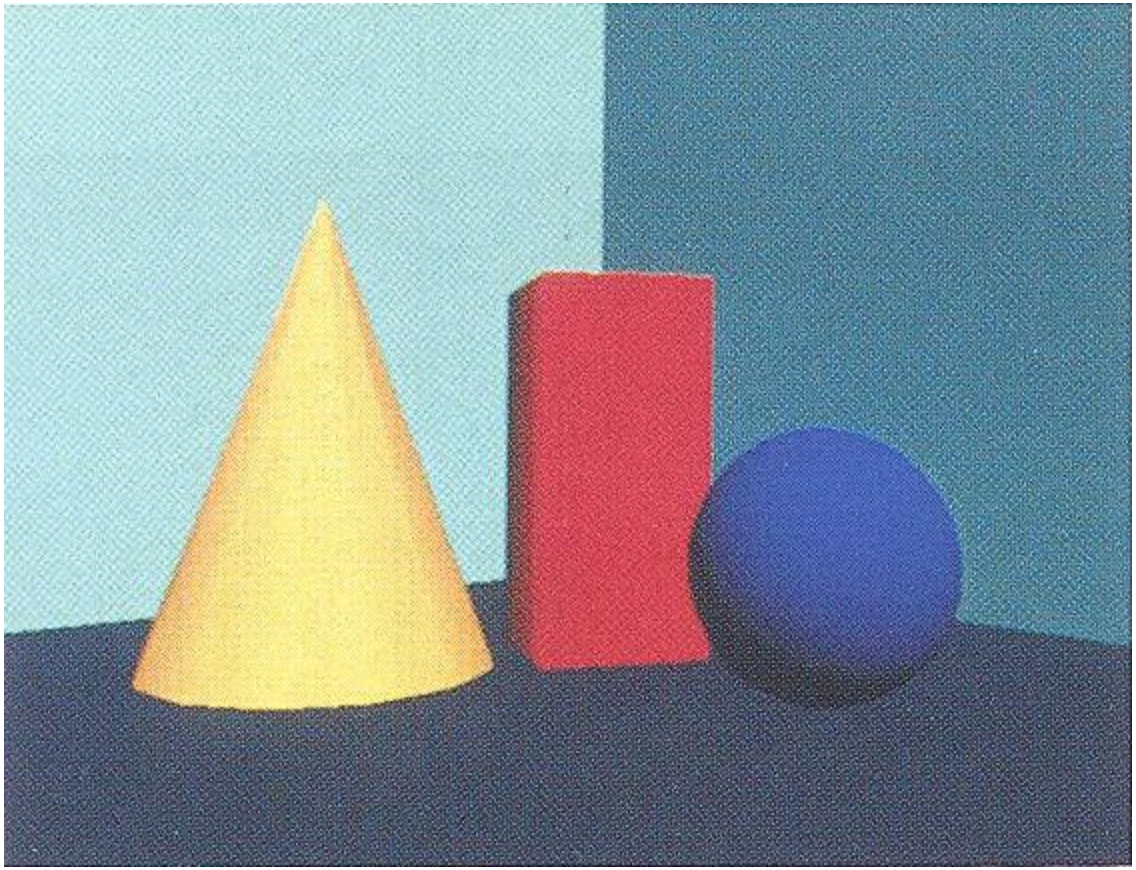


Light is scattered with equal intensity in all directions, **independent of the viewing direction**.

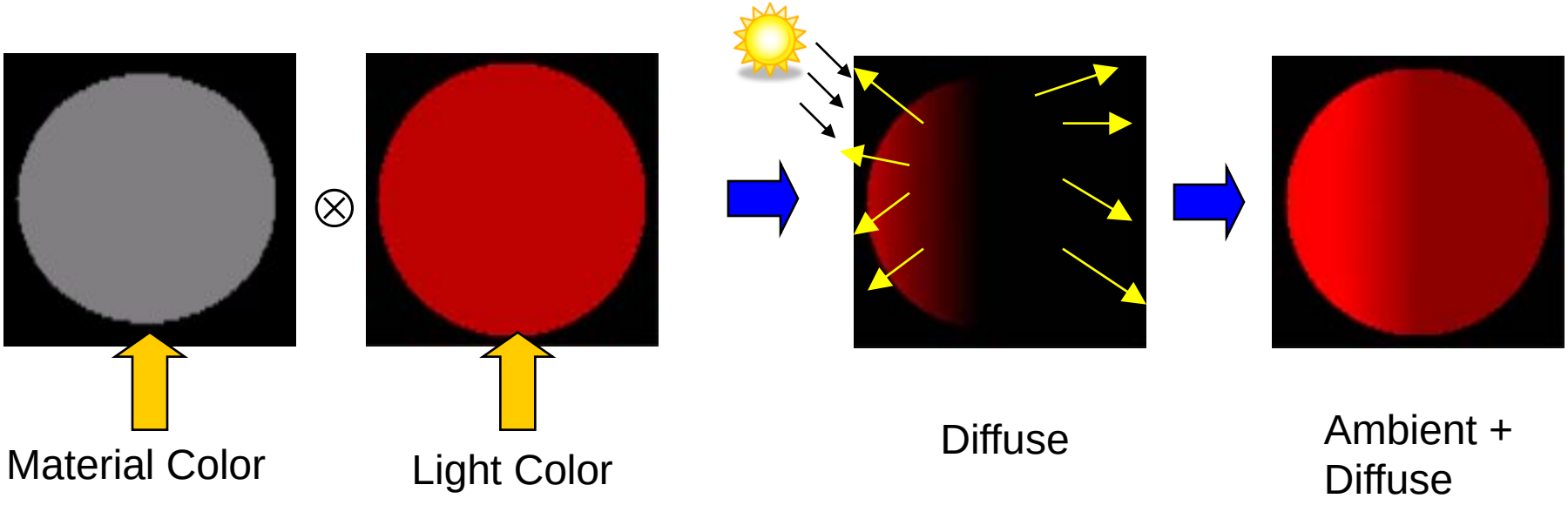
The surface appears equally bright from any viewing angle.

Ex: rough or grainy surfaces

Diffuse Lighting



Diffuse Lighting



Ambient + Diffuse

$$C_{diff} = M_{ambient} \otimes L_{ambient} + \max(-\vec{L} \bullet \vec{N}, 0) \cdot (M_{diff} \otimes L_{diff})$$

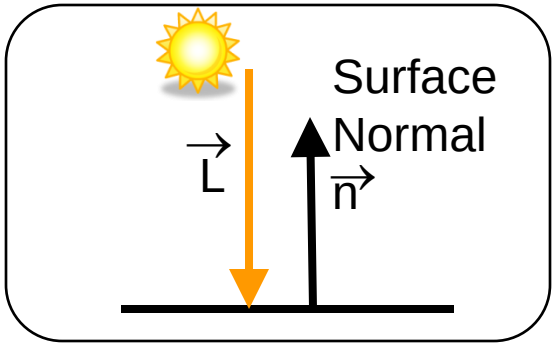
Diffuse Lighting

Background lighting with ambient light

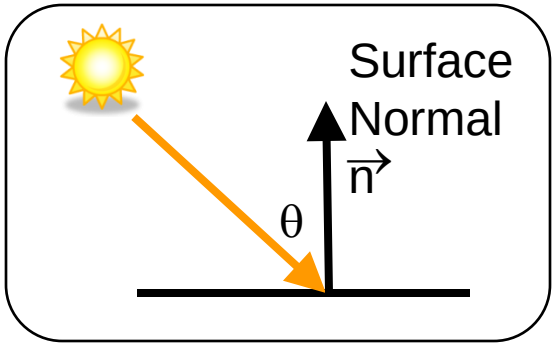
$$I_{ambdiff} = k_d \, I_a$$

k_d : Fraction of the incident light that is to be scattered as diffuse reflections (*diffuse reflection coefficient*)

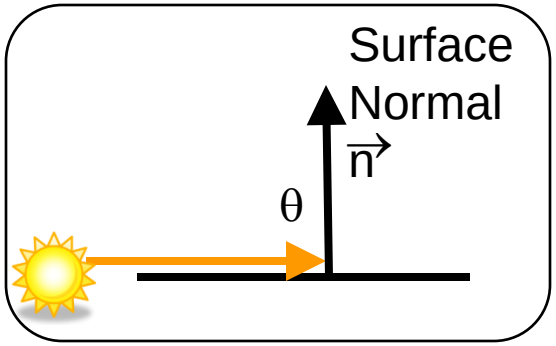
Diffuse Lighting



Full Reflection

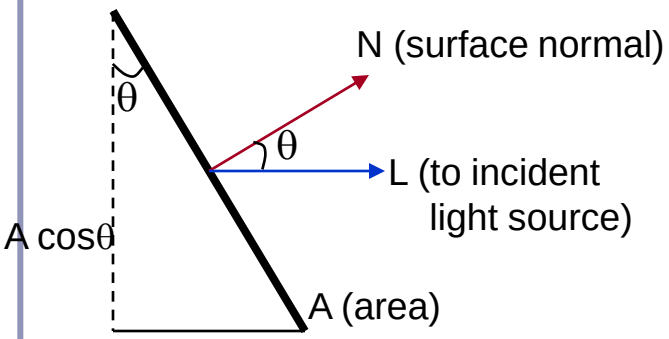


Partial



None

Diffuse Lighting



$\cos\theta = N.L$

$$I_{l,incident} = I_l \cos\theta$$

$$I_{l,diff} = k_d \,.\, I_{l,incident} = k_d \,.\, I_l \cos\theta$$

$$I_{l,diff} = \begin{cases} k_d \,.\, I_l\,(N.L) & \text{if } N.L > 0 \\ 0.0 & \text{if } N.L \le 0 \end{cases}$$

$$L = (P_{source} - P_{surf}) \,/\, |P_{source} - P_{surf}|$$

Ambient and point-source:

$$I_{diff} = \begin{cases} k_a \,.\, I_a + k_d \,.\, I_l\,(N.L) & \text{if } N.L > 0 \\ k_a \,.\, I_a & \text{if } N.L \le 0 \end{cases}$$

Diffuse Lighting

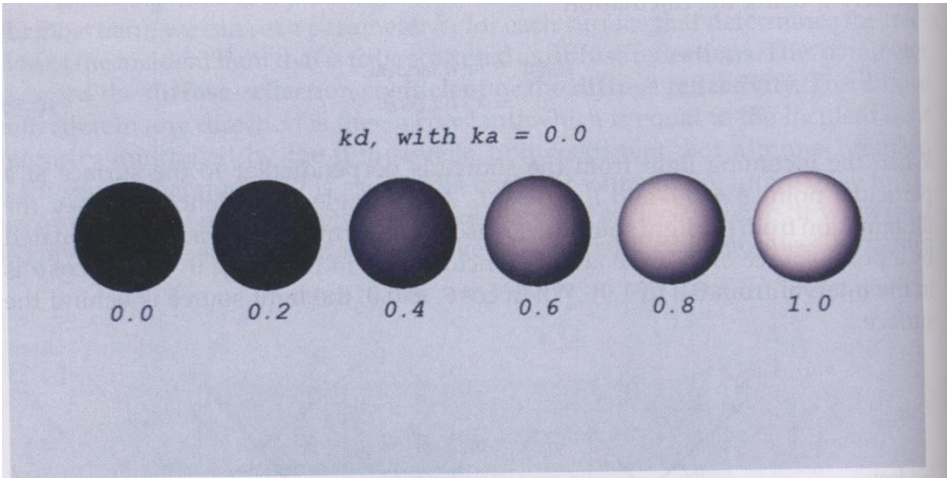


FIGURE 10-14 Diffuse reflections from a spherical surface illuminated by a point source emitting white light, with values of the diffuse reflectivity coefficient in the interval $0 \leq k_d \leq 1$.

Diffuse Lighting

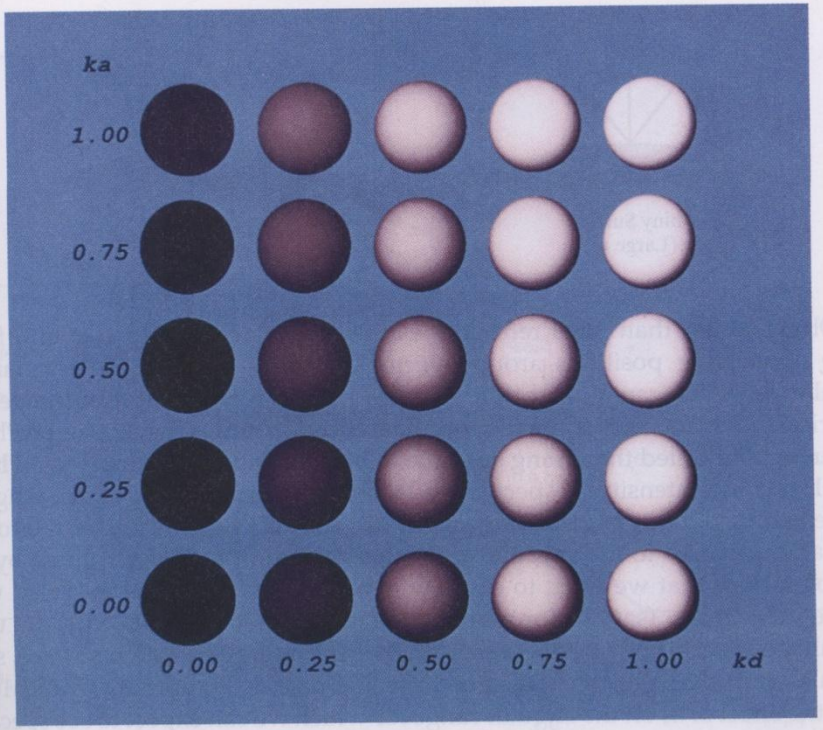
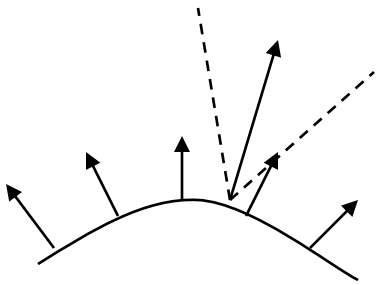


FIGURE 10-15 Diffuse reflections from a spherical surface illuminated with a dark-gray ambient light and a white point source, using five values for k_a and k_d from 0.0 to 1.0.

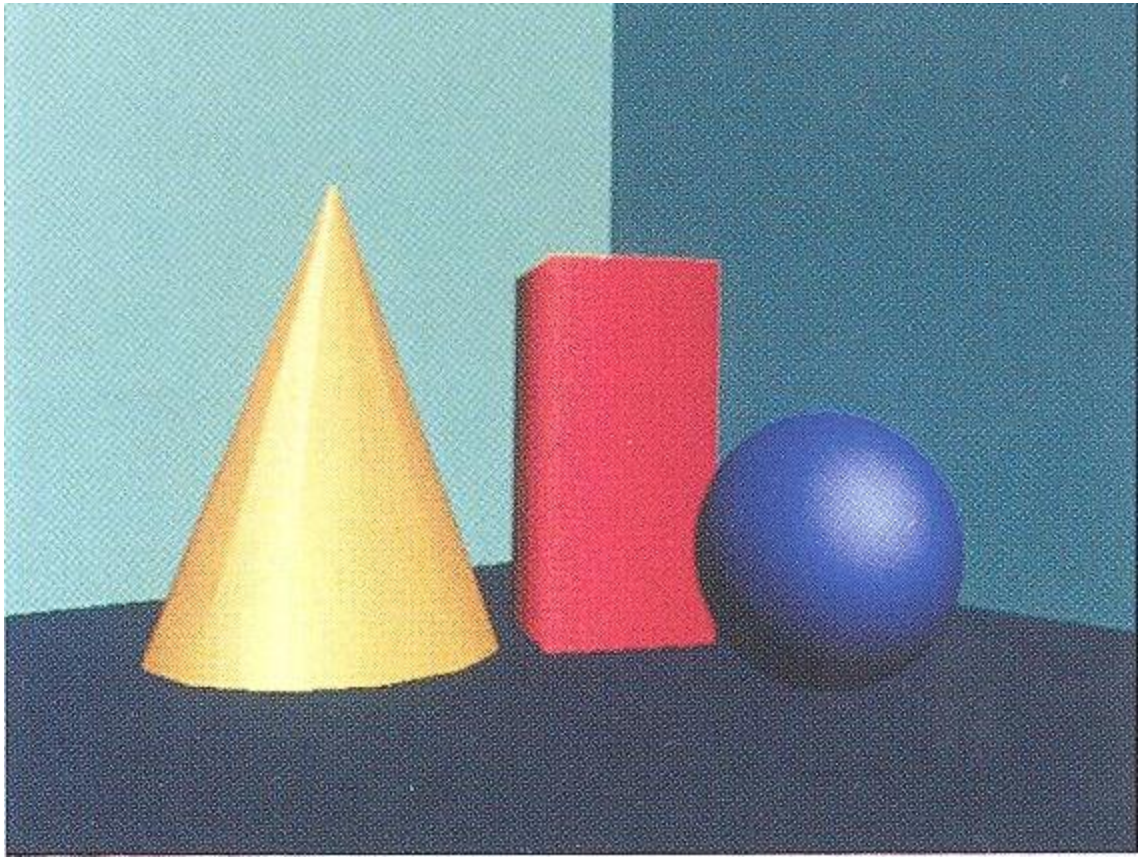
Specular Lighting



The incident light is reflected in a concentrated region around specular reflection angle.

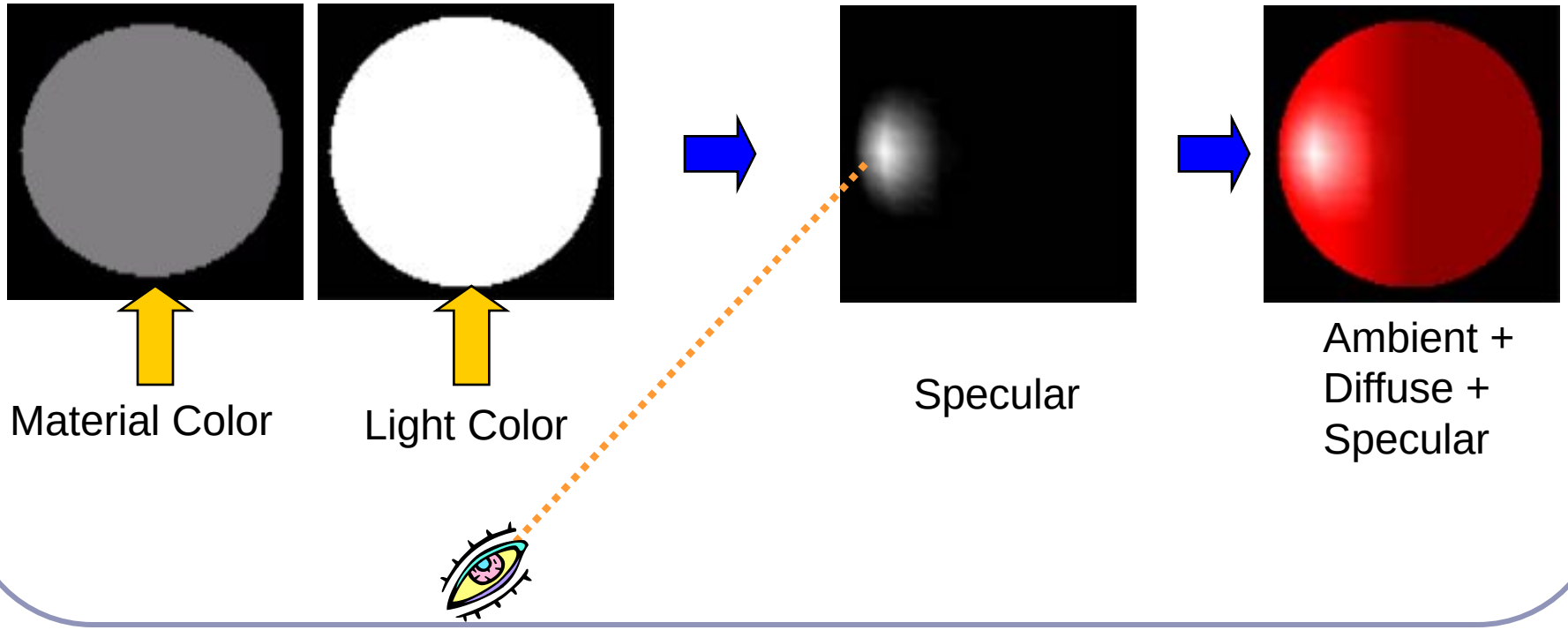
Ex: a bright spot on shiny surfaces

Specular Lighting

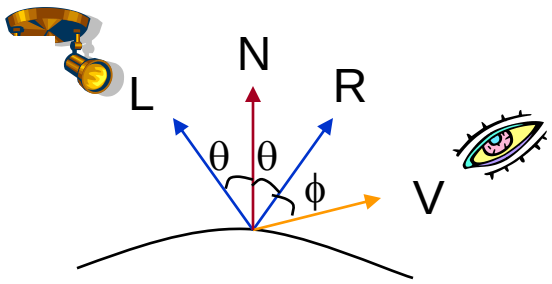


Specular Lighting

- Create shining surface (surface perfectly reflects)
- **Viewpoint dependent**



Specular Lighting



N – surface normal

L – to light source

R – direction of ideal specular reflection

V – to viewer

ϕ – viewer angle relative to specular reflection direction

if ϕ=0, ideal reflector (mirror)

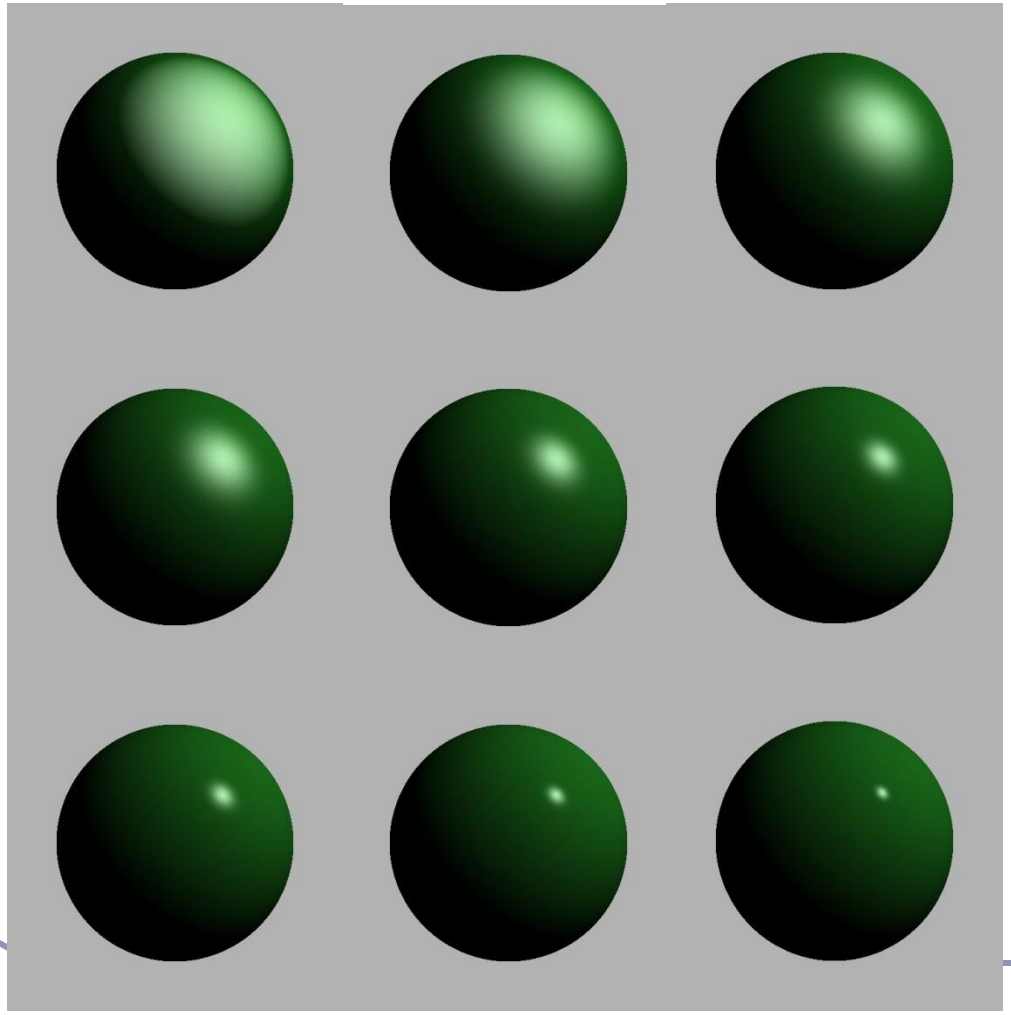
Specular Lighting

Sets the intensity of specular reflection proportional to $\cos^{ns}\phi$.

n_s: specular reflection exponent

100 or more	=> shiny
near 1	=> dull
infinite	=> perfect reflector

Specular Lighting



- A larger n_s shows more concentration of the reflection

Specular Lighting

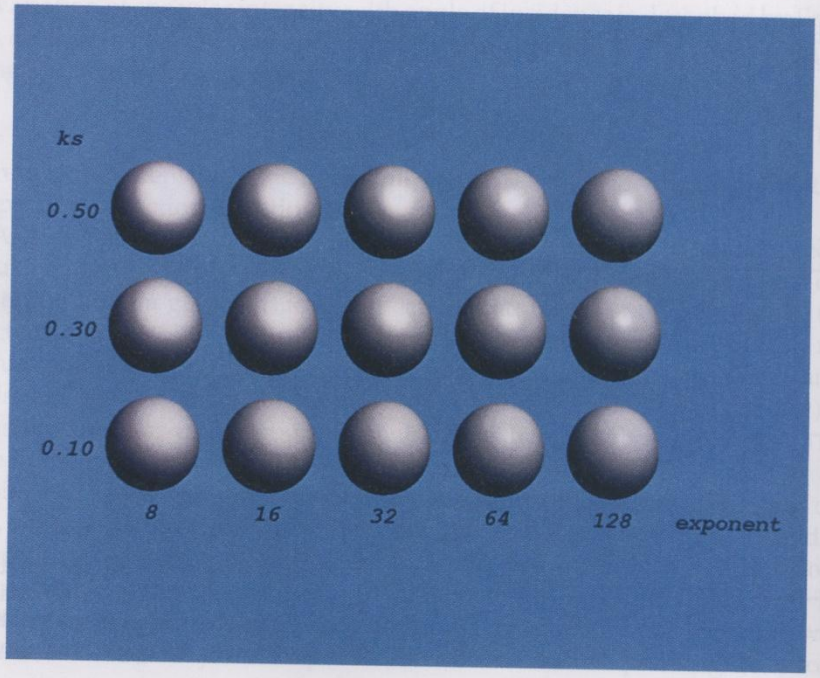
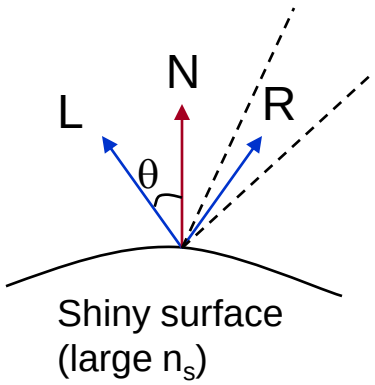
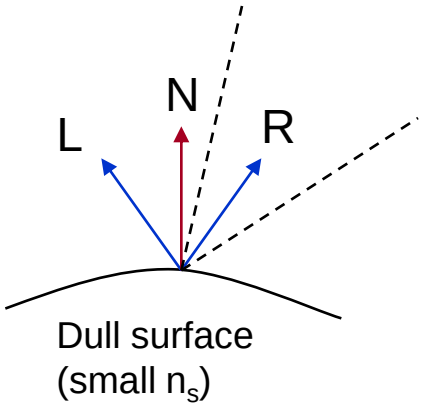


FIGURE 10-21 Specular reflections from a spherical surface for varying specular parameter values and a single light source.

Phong Model





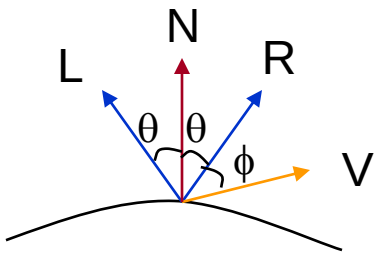
W(θ): specular reflection coefficient of the surface

$I_{l,spec} = W(\theta)\, I_l \cos^{ns}\phi$

opaque surface: W(θ)=constant=k_s (0.0-1.0)

transparent surface: reflection is appreciable if θ approaches to 90^0.

Phong Model



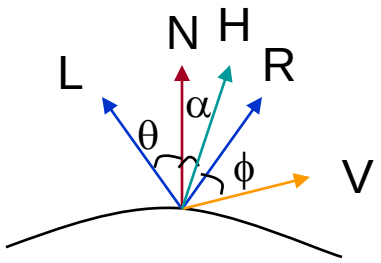
$$\cos\phi = V \cdot R$$

$$I_{l,spec} = W(\theta)\, I_l \cos^{ns}\phi$$

$$I_{l,spec} = \begin{cases} k_s\, I_l\, (V.R)^{ns} & \text{if } V.R>0 \;\&\; N.L>0 \\ 0.0 & \text{if } V.R<0 \;\&\; N.L\leq 0 \end{cases}$$

$$R = (2N.L)N - L$$

Phong Model



H: halfway vector between V and L

$H=(L+V) / |L+V|$

Replace V.R ($\cos\phi$) by N.H ($\cos\alpha$)

$$I_{l,spec} = \begin{cases} k_s\, I_l\, (N.H)^{ns} & \text{if } V.R>0 \;\&\; N.L>0 \\ 0.0 & \text{if } V.R<0 \;\&\; N.L\leq 0 \end{cases}$$

Requires less computation

Lighting

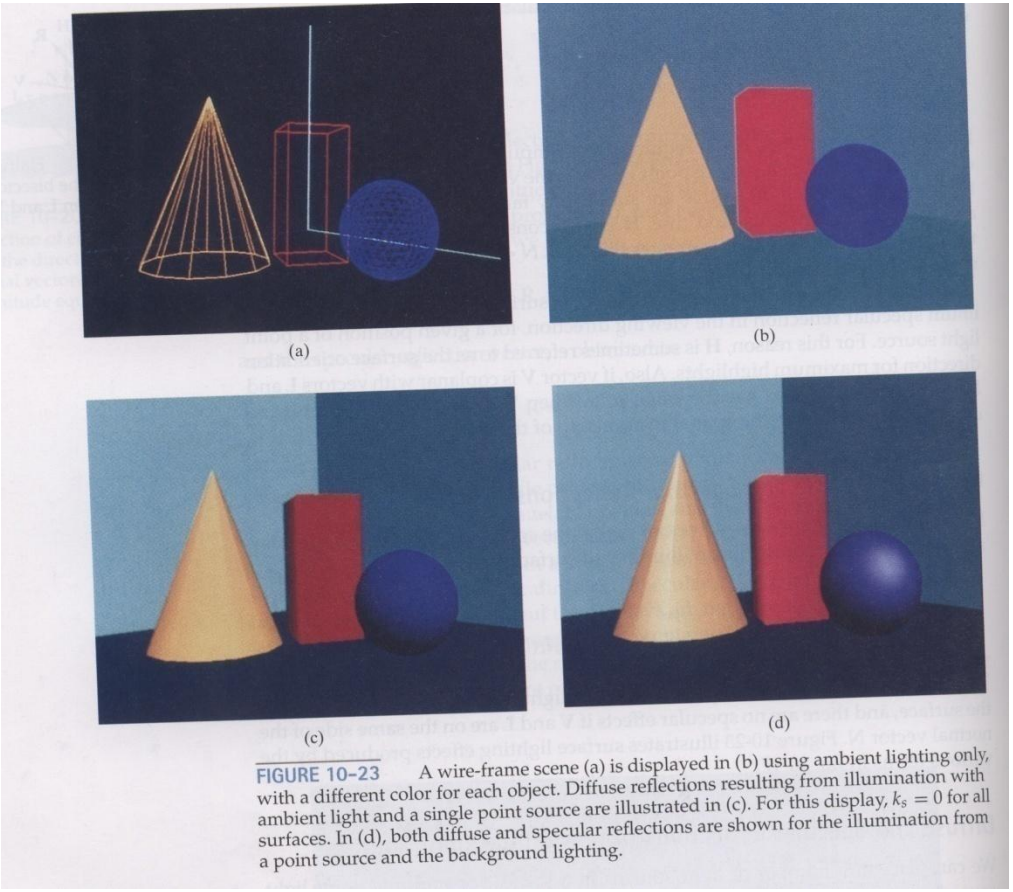


FIGURE 10-23 A wire-frame scene (a) is displayed in (b) using ambient lighting only, with a different color for each object. Diffuse reflections resulting from illumination with ambient light and a single point source are illustrated in (c). For this display, $k_s = 0$ for all surfaces. In (d), both diffuse and specular reflections are shown for the illumination from a point source and the background lighting.

Diffuse and Specular Lightings

$$I = I_{diff} + I_{spec}$$

$$= k_a I_a + k_d I_l (N.L) + k_s I_l (N.H)^{ns}$$

Multiple Light Sources

$$I = I_{ambdiff} + \sum^{n}_{l=1} (I_{l,diff} + I_{l,spec})$$

$$= k_a I_a + \sum^{n}_{l=1} I_l (k_d(N.L) + k_s(N.H)^{ns})$$

Surface Light Emission

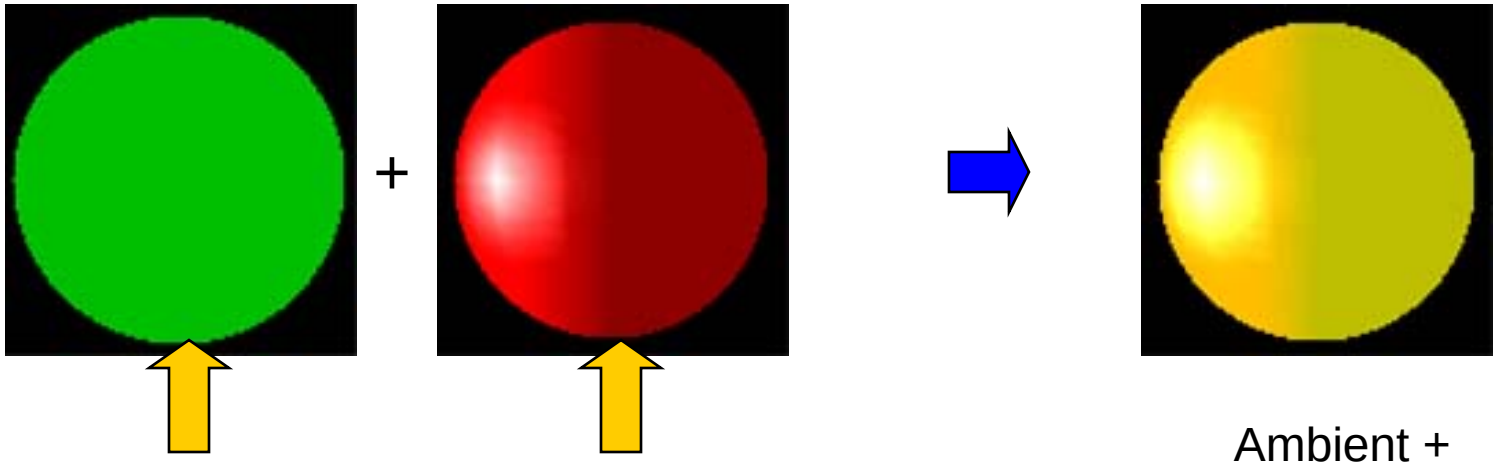
Ex: light bulb, etc.

- Position a directional light source behind the surface
- Simulate the emission with a set of point light sources distributed over the surface

Radiocity model can be used to model realistic surface emission.

Surface Light Emission

- Color is emitted by the **material** only



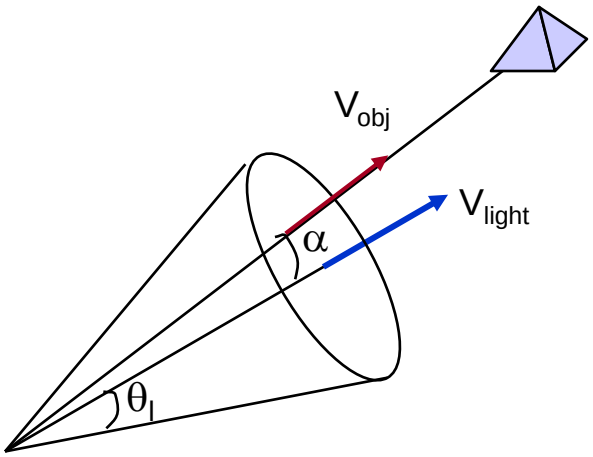
Material
Emissive
Color

Ambient +
Diffuse +
Specular

Ambient +
Diffuse +
Specular +
Emissive

$$C_{all} = C_e + M_a \otimes L_a + \max(-\vec{L} \bullet \vec{N}, 0) \cdot (M_d \otimes L_d) + (\max(\vec{N} \bullet \vec{H}, 0))^{ns} \cdot M_s \otimes L_s$$

Directional Light Sources



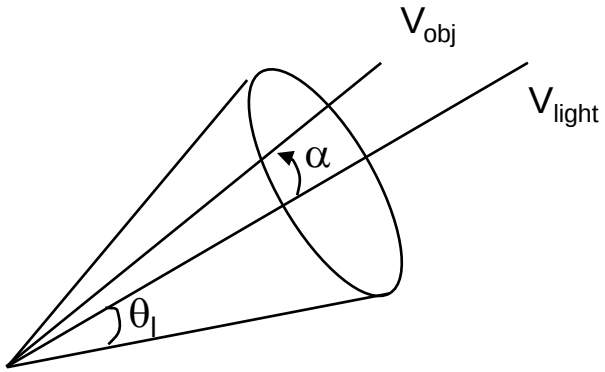
$0^0 < \theta \leq 90^0$

Object is within the spot light if

$\cos \alpha \geq \cos \theta_l$

$V_{obj} \cdot V_{light} = \cos \alpha$

Angular Intensity Attenuation



$$f_{angatten}(\alpha) = \cos^a \alpha$$

$$f_{l,angatten} = \begin{cases} 1.0 & \text{if light source is not spotlight} \\ 0.0 & \text{if } V_{obj} \cdot V_{light} = \cos\alpha < \cos\theta_l \\ & \text{(outside of spotlight cone)} \\ (V_{obj} \cdot V_{light})^{al} & \text{otherwise} \end{cases}$$

Surface Light Emission

Extending the Light Model:

$$I = I_{surfemission} + I_{ambdiff} + \sum_{l=1}^{n} f_{l,radatten}\, f_{l,angatten}\, (I_{l,diff} + I_{l,spec})$$

$$f_{l,radatten}(d_l) = \begin{cases} 1/(a_0 + a_1 d_l + a_2 d_l^2) & \textit{local} \\ 1.0 & \textit{infinity} \end{cases}$$

$$I_{l,diff} = \begin{cases} k_d \cdot I_l\,(N.L_l) & \text{if } N.L > 0 \\ 0.0 & \text{if } N.L \le 0 \end{cases}$$

$$f_{l,angatten} = \begin{cases} 1.0 & \text{if light source is not spotlight} \\ 0.0 & \text{if } V_{obj}.V_{light} = \cos\alpha \; < \cos\theta_l \\ (V_{obj} \cdot V_{light})^{al} & \text{otherwise} \end{cases}$$

$$I_{l,spec} = \begin{cases} k_s\, I_l \max\{0.0, (N.H_l)^{ns}\} & \text{if } V.R>0 \;\&\; N.L>0 \\ 0.0 & \text{if } V.R<0 \;\&\; N.L\le 0 \end{cases}$$

Surface Light Emission

Intensity Overflow:

- If any term exceeds the maximum intensity, set the intensity to maximum.
- Divide the intensity if each term by the magnitude of the largest term to scale intensities between 0.0-1.0
- Set negative intensities to 0.0.

RGB Color

$I_l = (I_{lR}, I_{lG}, I_{lB})$

$k_a = (k_{aR}, k_{aG}, k_{aB})$

Ex: for blue component

$I_{lB,diff} = k_{dB}\, I_{lB}\, (N.L_l)$

$I_{l,spec} = k_{sB}\, I_{lB}\, \max\{0.0, (N.H_l)^{ns}\}$

Luminance

Lightness or darkness of a color (Brightness)

$$\text{luminance} = \int_{\text{visible f}} \rho(f)\, I(f)\, df$$

f: frequency

$I(f)$: intensity of light component with frequency f that is radiating in a particular direction

$\rho(f)$: experimentally determined proportionality function that varies with f and illumination level

Green component contribute **most** to the luminance

Blue component contribute **least** to the luminance

luminance = 0.299 R + 0.587 G + 0.114 B or

luminance = 0.2125 R + 0.7154 G + 0.0721 B

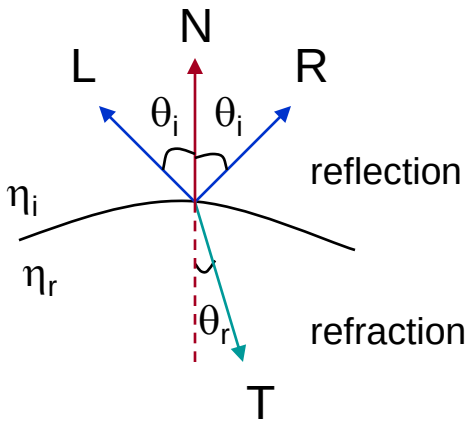
Transparency

Transparent: objects behind are seen
Opaque: objects behind cannot be seen
Translucent: objects behind appear blurred

Transparent surfaces produce **reflected** and **transmitted** light.

Translucent surfaces have both **diffuse** and **specular** transmission. Ray tracing can be used.

Light Refraction



Snell's Law:

$\sin \theta_r = (\eta_i/\eta_r) \sin \theta_i$

θ_i: angel of incidence

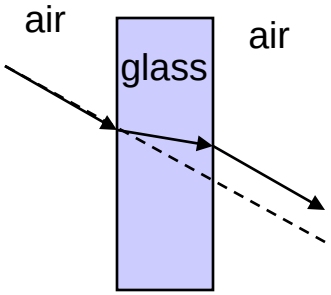
η_i: index of refraction for the incident material

η_r: index of refraction for the refracting material

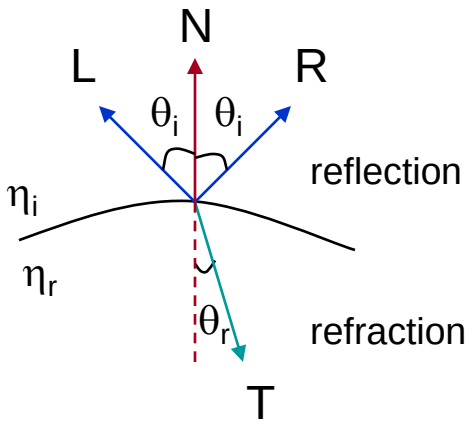
Ex:

air: 1.0

glass: 1.61



Light Refraction

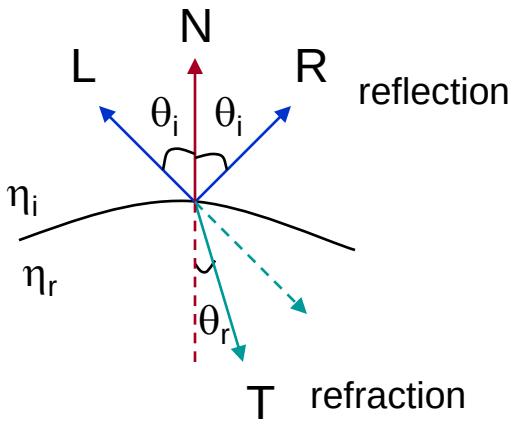


$$T = [(\eta_i/\eta_r)\cos\theta_i - \cos\theta_r]\,N - (\eta_i/\eta_r)L$$

Gives realistic effect but requires calculation

$\Rightarrow$ Ray tracing

Light Refraction



Ignoring path shift due to refraction:

$$I = (1-k_t)\, I_{refl} + k_t\, I_{trans}$$

I_{trans}: transmitted intensity

I_{refl}: reflected intensity

k_t: transparency coefficient

- 1.0 – transparent
- 0.0 – opaque

$(1-k_t)$: opacity factor

Light Refraction

In Depth-Buffer Approach:

- Process opaque surfaces first
- Then, compare the depths of the transparent surfaces with the depth-buffer
- If a transparent surface is visible, combine its intensity with the intensity that is previously stored in the frame buffer.

Light Refraction

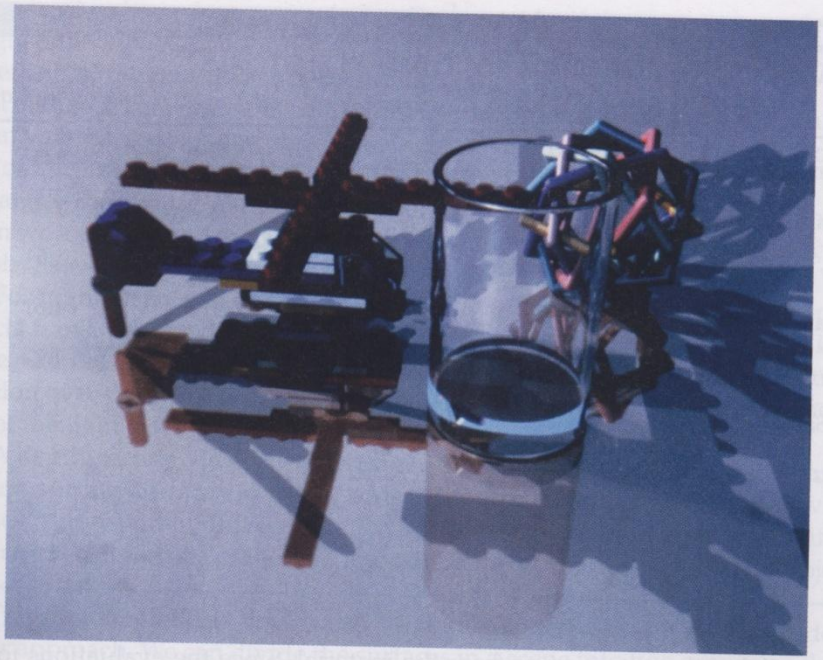


FIGURE 10-29 A ray-traced view of a scene containing a transparent drinking glass, showing both light transmissions from objects behind the glass and light reflections from the glas surface. (*Courtesy of Eric Haines, Autodesk, Inc.*)

Light Refraction

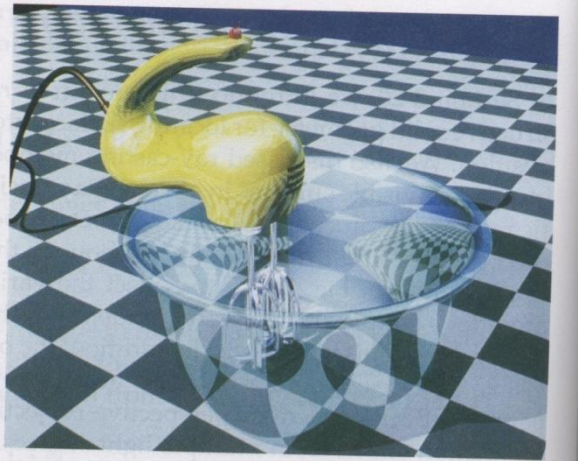


FIGURE 10-53 A ray-traced scene, showing global reflection and transparency effects. (*Courtesy of Evans & Sutherland.*)

Atmospheric Effects

$f_{atmo}(d) = e^{-\rho d}$ or $f_{atmo}(d) = e^{-(\rho d)_2}$

d: distance of the object to the viewing position

ρ: to set a positive density value for the atmosphere

Higher value means **denser** atmosphere, where colors are **muted**.

Combining atmosphere color with the object's color:

$$I = f_{atmo}(d)\, I_{obj} + (1 - f_{atmo}(d))\, I_{atmo}$$

Shadows

- Generated by a visible surface detection method
- Surfaces that are visible from the view position are shaded according to the lighting model, which can be combined with texture patterns
- Shadow areas are displayed with ambient light only or ambient light combined with textures

Polygon Rendering

Constant-Intensity Surface Rendering

Accurate in the following cases:

- Polygon is not a section of a curved surface
- All light sources are sufficiently far from the surface (N.L is constant)
- Viewing position is sufficiently far from the surface (V.R is constant)

Polygon Rendering

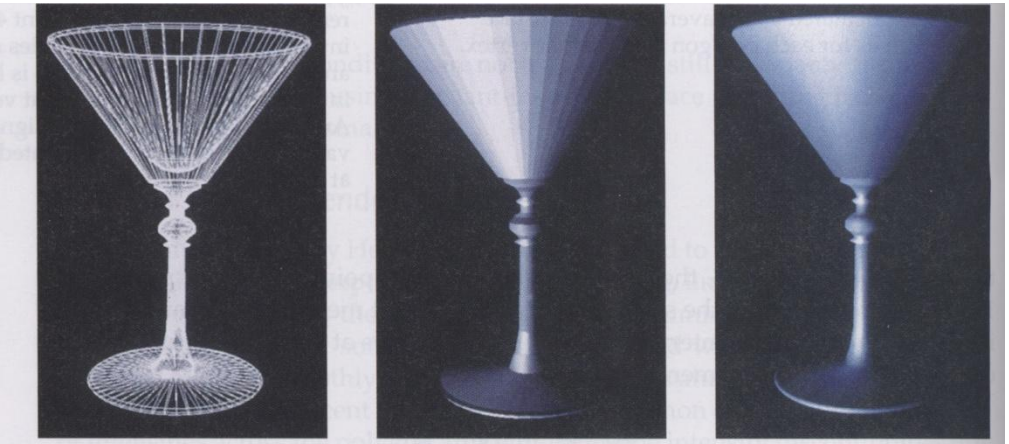


FIGURE 10-50 A polygon mesh approximation of an object (a) is displayed using flat surface rendering in (b) and using Gouraud surface rendering in (c).

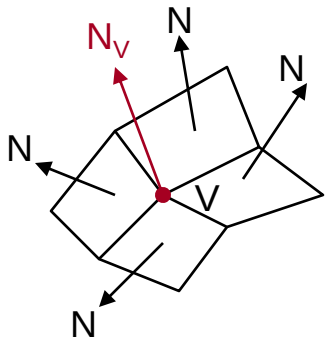
Polygon Rendering

Gouraud Surface Rendering

Linearly interpolates vertex intensities across the polygon faces.

1. Determine the average unit normal vector at each vertex of the polygon
2. Apply an illumination model at each polygon vertex to obtain the light intensity at that position
3. Linearly interpolate the vertex intensities over the projected area of the polygon

Polygon Rendering

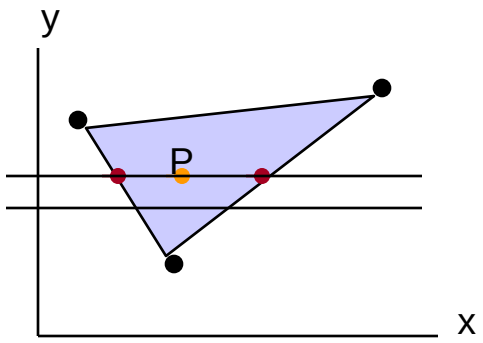


Gouraud Surface Rendering

(1) At each vertex, average the normal vectors of the polygons that share the same vertex, to obtain the normal vector of the vertex:

$$N_V = \frac{\sum_{k=1}^{n} N_k}{\left| \sum_{k=1}^{n} N_k \right|}$$

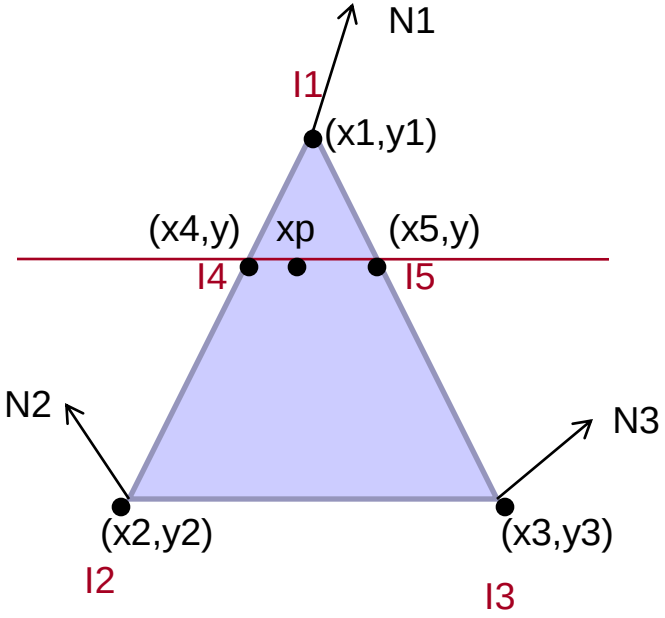
Polygon Rendering



Gouraud Surface Rendering

(3) Interpolate vertex values to obtain the intensities at the intersection of a scan line with the polygon

Polygon Rendering



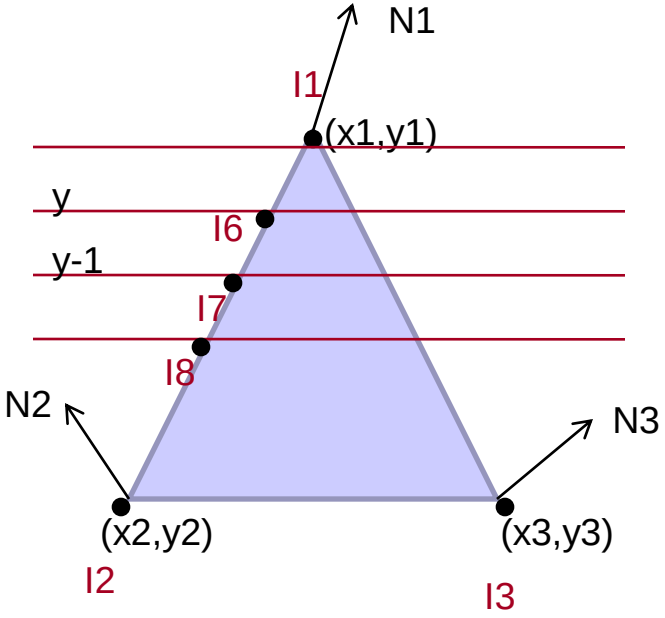
Gouraud Surface Rendering

$$I_4 = \frac{y - y_2}{y_1 - y_2} I_1 + \frac{y_1 - y}{y_1 - y_2} I_2$$

$$I_5 = \frac{y - y_3}{y_1 - y_3} I_1 + \frac{y_1 - y}{y_1 - y_3} I_3$$

$$I_p = \frac{x_5 - x_p}{x_5 - x_4} I_4 + \frac{x_p - x_4}{x_5 - x_4} I_5$$

Polygon Rendering



Gouraud Surface Rendering

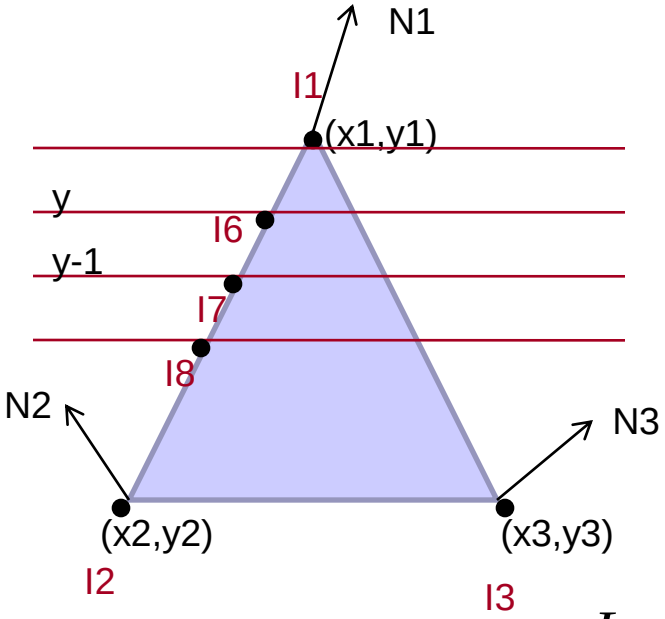
at y:

$$I_6 = \frac{y_1 - y}{y_1 - y_2} I_2 + \frac{y - y_2}{y_1 - y_2} I_1$$

$$I_6 = \frac{1}{y_1 - y_2} I_2 + \frac{y_1 - y_2 - 1}{y_1 - y_2} I_1$$

$$I_6 = I_1 + \frac{I_2 - I_1}{y_1 - y_2}$$

Polygon Rendering



Gouraud Surface Rendering

at y:

$$I_6 = \frac{y_1 - y}{y_1 - y_2} I_2 + \frac{y - y_2}{y_1 - y_2} I_1$$

at y-1:

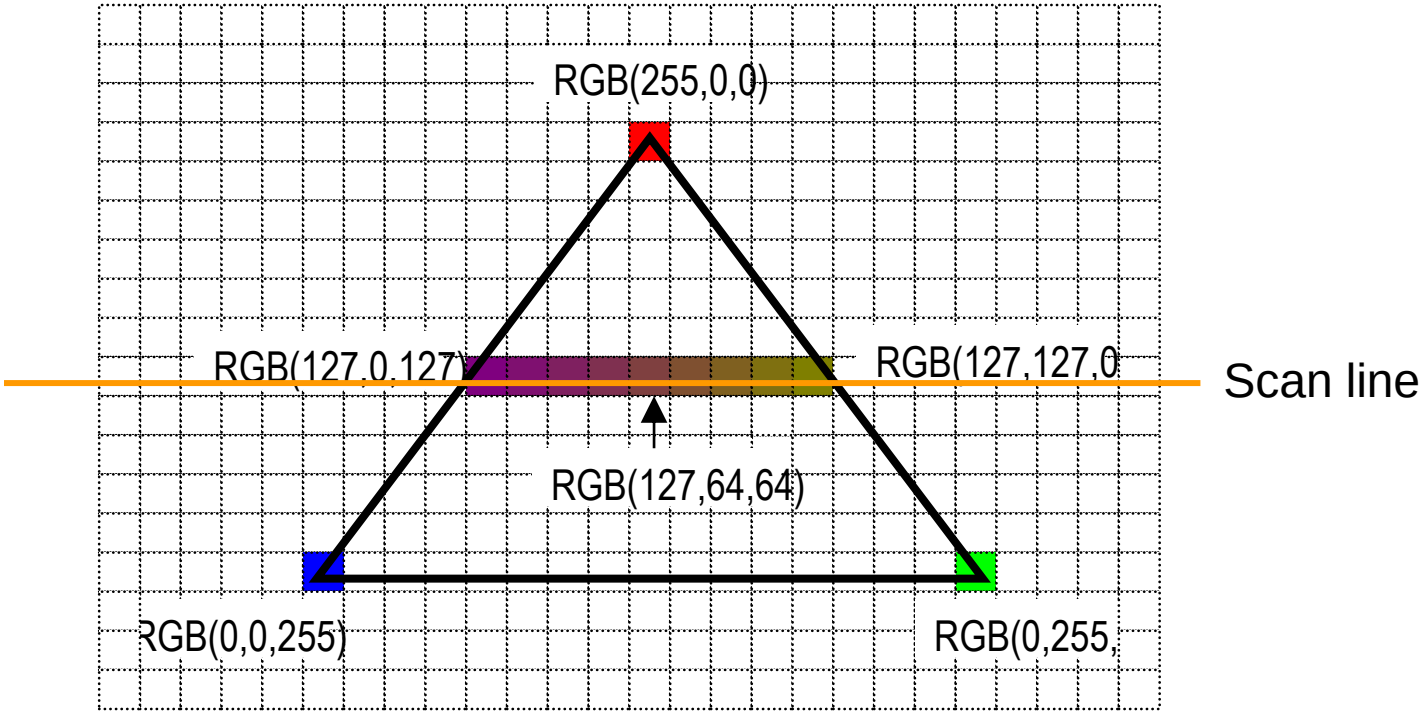
$$I_7 = \frac{(y-1) - y_2}{y_1 - y_2} I_1 + \frac{y_1 - (y-1)}{y_1 - y_2} I_2$$

$$I_7 = \frac{(y - y_2)}{y_1 - y_2} I_1 - \frac{1}{y_1 - y_2} I_1 + \frac{(y_1 - y)}{y_1 - y_2} I_2 + \frac{1}{y_1 - y_2} I_2$$

$$I_7 = I_6 + \frac{I_2 - I_1}{y_1 - y_2}$$

$$I_7 = I_1 + 2\frac{I_2 - I_1}{y_1 - y_2}$$

Polygon Rendering



Scan conversion algorithm

Polygon Rendering

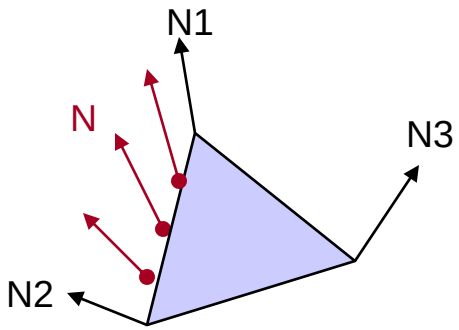
Phong Surface Rendering

For each polygon:

1. Determine the average unit normal vector at each vertex of the polygon
2. Linearly interpolate the vertex normals over the projected area of the polygon (similar to Gouraud method)
3. Apply an illumination model at positions along scan lines to calculate pixel intensities using the interpolated normal vectors

Apply the same incremental methods to obtain normal vectors on successive scan lines.

Polygon Rendering

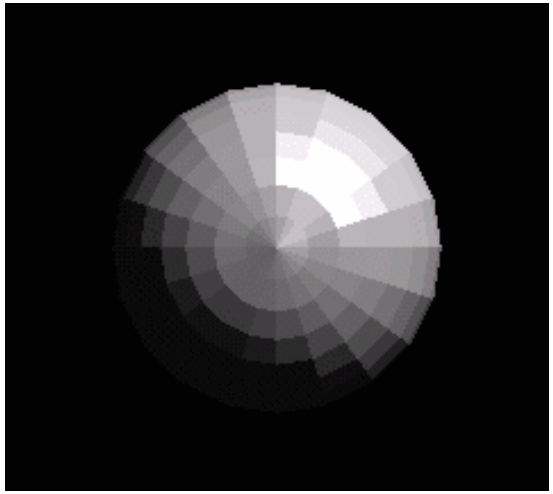


Phong Surface Rendering

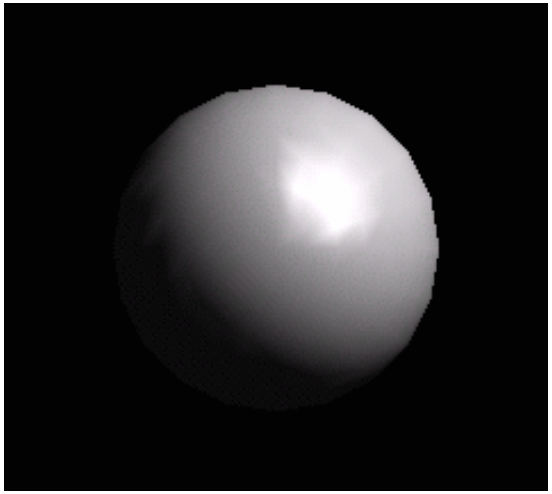
Interpolates normal vectors.

Polygon Rendering

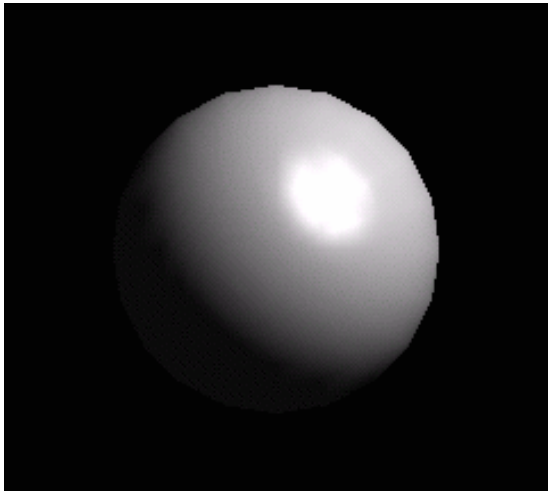
Source: Michal Necasek



Flat shading



Gouraud shading



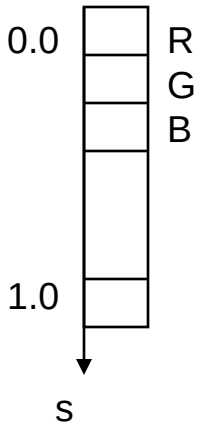
Phong shading

Texture Mapping

Texel: a component of texture description

Four RGB components:
- Red component
- Green component
- Blue component
- an index into a color table *or* a single luminance value

Texture Mapping

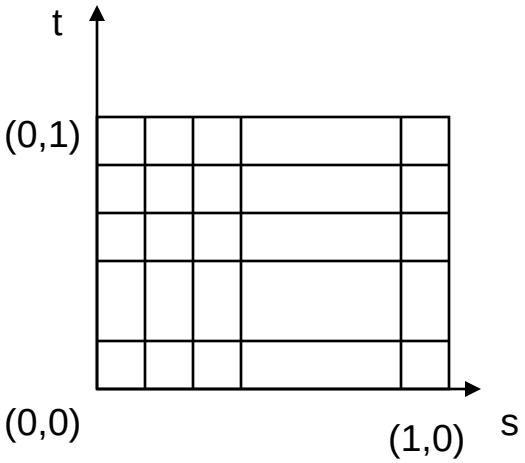


3 x n elements

Linear Texture Patterns (1D)

s - coordinate texture space

Texture Mapping



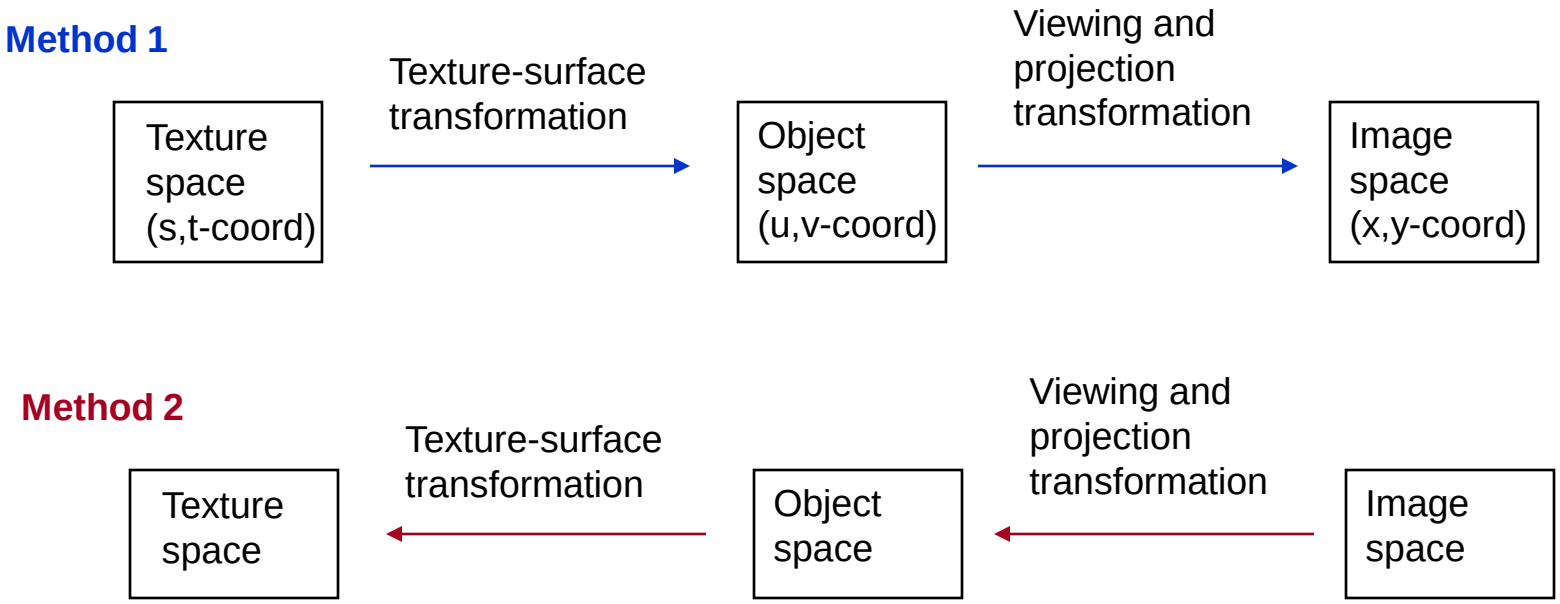
Surface Texture Patterns (2D)

s,t - coordinate texture space

3 x m x n elements

Texture Mapping

Surface Texture Patterns (2D)



Texture Mapping

Surface Texture Patterns (2D)

Method 1:

Mapping from Texture Space to Object Space –

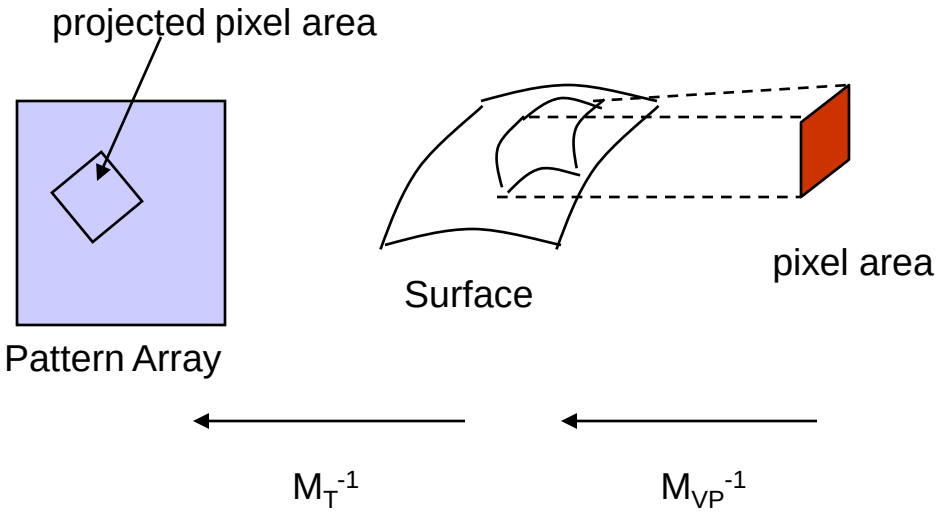
$u = u(s,t) = a_u\, s + b_u\, t + c_u$

$v = v(s,t) = a_v\, s + b_v\, t + c_v$

Mapping from Object Space to Image Space – use viewing and projection transformations

Disadvantage: If the texture patch does not match up with the pixel boundaries, the fractional area of pixel coverage must be calculated.

Texture Mapping



Surface Texture Patterns (2D)

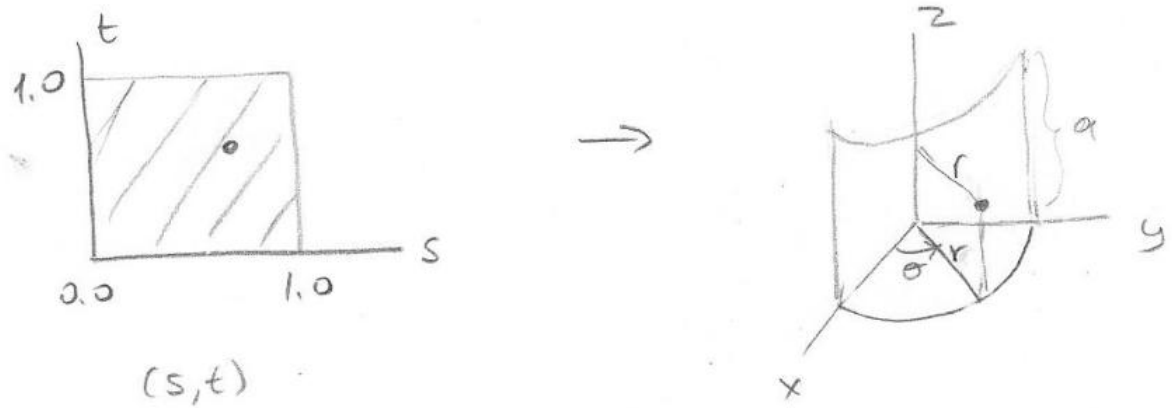
Method 2:

M_{VP}^{-1}: inverse viewing-projection transformation

M_T^{-1}: inverse texture-map transformation

Texture Mapping

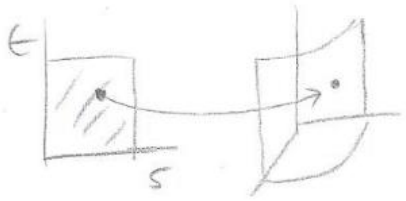
Ex:



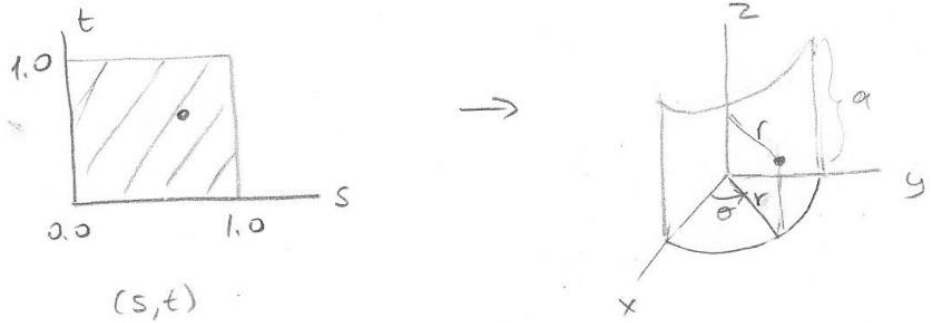
Texture Mapping

Ex: Method 1

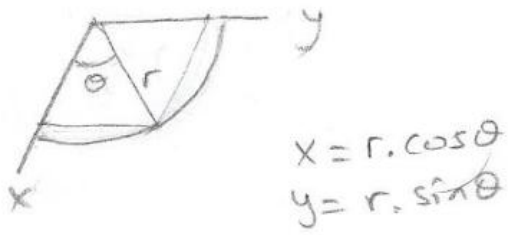
Texture pattern (s,t) → Object surface (x,y,z)



(s,t)	(u,v)	(x,y,z)
0≤ s ≤1 0≤ t ≤1	u=θ 0≤ θ ≤ Π/2	



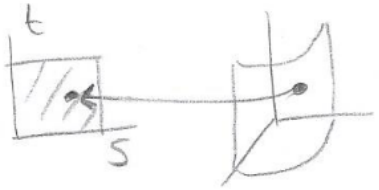
(s,t)→ (u,v)	(u,v)→(x,y,z)
u = s.Π/2 v = a.t	x = r.cos u y = r.sin u z = v

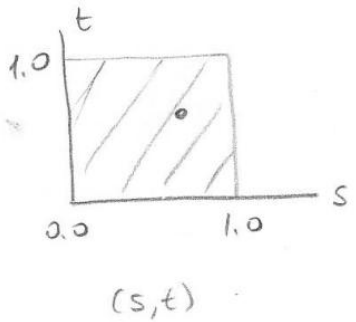


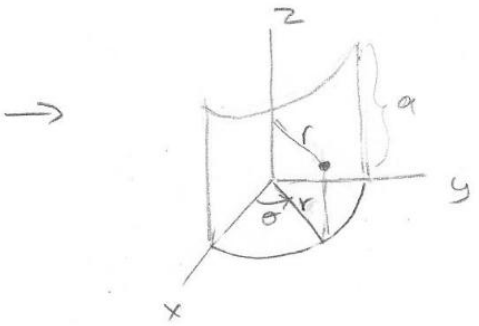
Texture Mapping

Ex: Method 2

Object surface (x,y,z) → Texture pattern (s,t)

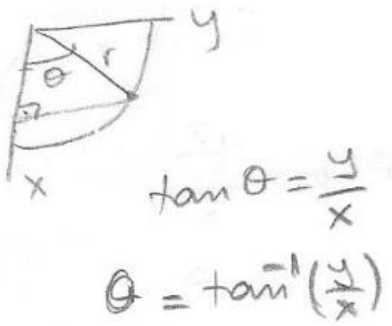






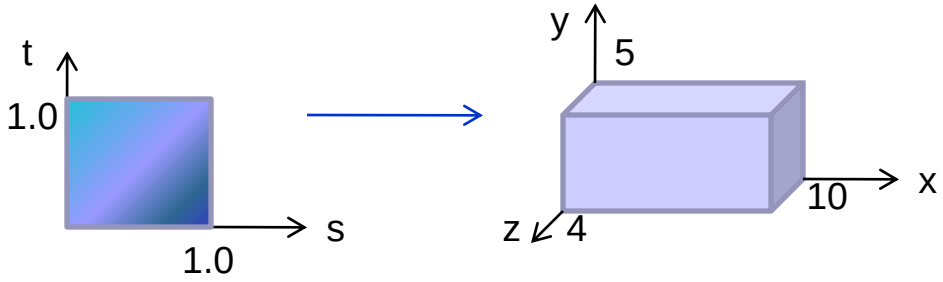
(x,y,z)	(u,v)	(s,t)
	$u=\theta$ $0 \le u \le \Pi/2$	$0 \le s \le 1$ $0 \le t \le 1$

(x,y,z)→ (u,v)	(u,v)→(s,t)
$u = \tan^{-1}(y/x)$ $v = z$	$s = u.\Pi/2$ $t = v/a$



Texture Mapping

Ex: Method 1

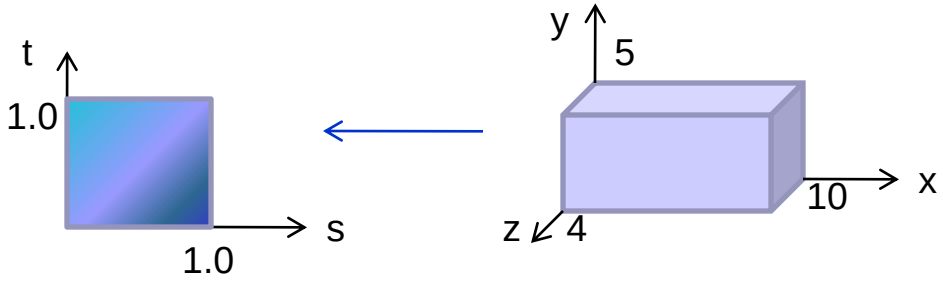


(s,t)→ (u,v)	(u,v)→(x,y,z)
u = s v = t	x = 10 u y = 5 v z = 4

(0.8, 0.6) → (8,3,4)

Texture Mapping

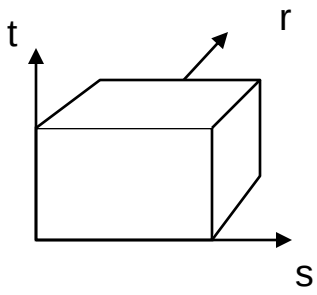
Ex: Method 2



(8,3,4) → (0.8, 0.6)

(x,y,z)→ (u,v)	(u,v)→(s,t)
u = x/10 v = y/5	s = u t = v

Texture Mapping



3 x m x n x k elements

Volume Texture Patterns (3D)

s,t,r - coordinate texture space